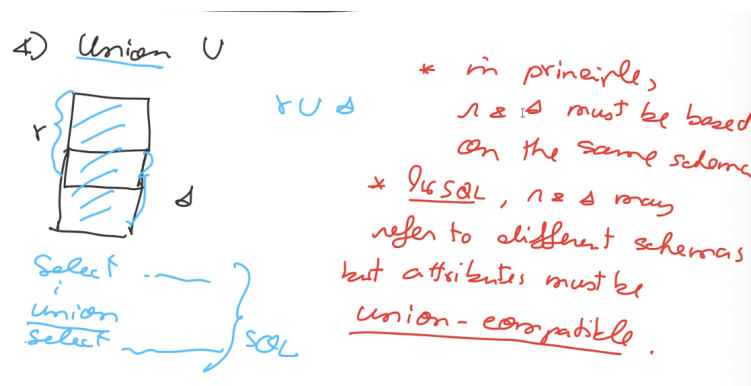




The remaining relational algebra.

✓ Union

4. Union



- (relational algebra) in principle, r and s must be based on the same schema.
 - the attribute must be the same. $\rightarrow$ to make sense
- in sql, r and s may refer to different schemas, but attribute must be union compatible.
 - union compatible column like character with character, you might not be able to union like char and int.
 - this union-compatibility property depends on sbms.

✓ Intersection

5. Intersection

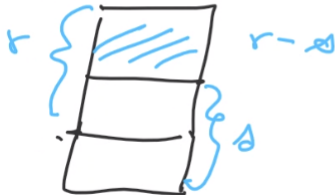


- the concept of union-compatible is also applies

✓ Difference (Minus)

6. difference

6) Difference (Minus)



✓ Product (the cartesian product)

$r \times s$

✓ Divide

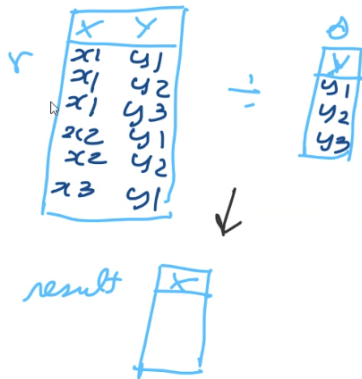
The dividend r is a binary relation, the divider s is a unary relation (1 column).

8) Divide $r \div s$ The dividend r is a binary relation.

The divider s is a unary relation.

The result is a unary relation whose attribute is not the attribute of the divider and whose values are the values that match all the values of the divider.

- for example 1,
- whose attribute is not the attribute of divisor
 - have to be x
- and the vales that match all the values of the divisor



- result: x_1 (the only value that has the map to y_1 , y_2 , and y_3)

if we change s to have only y_1 , $y_2 \rightarrow$ result will be both x_1 , x_2

if we change s to have only $y_1 \rightarrow$ result will be all x_1 , x_2 , x_3 .

Application of this divide

Data Structure:

- Domains (atomic values)
- n -ary relations (attributes, tuples)

Data -integrity:

1. primary key values must not be null
2. foreign key values must match primary key values (or be null)

Data manipulation:

- Relational algebra (or relational calculus equivalents):
 - Union, intersection, difference, product
 - Select, project, join, divide
 - Relational assignment
- The relational model (in outline).

- data manipulation
 - has to be as powerful as relational algebra or relational calculus.

- sql is equivalent
- to prove, is to be able to show whether sql can do the divide operation
- the typical query
 - very useful for query that have the keyword all.

S	S #	SNAME	STATUS	CITY	SP	S #	P #	QTY
	S 1	SMITH	20	LONDON		S 1	P 1	300
	S 2	JONES	10	PARIS		S 1	P 2	200
	S 3	BLAKE	30	PARIS		S 1	P 3	400
	S 4	CLARK	20	LONDON		S 1	P 4	200
	S 5	ADAMS	30	ATHENS		S 1	P 5	100
P	P #	PNAME	COLOR	WEIGHT	CITY			
	P 1	Nut	Red	12	London	S 1	P 6	100
	P 2	Bolt	Green	17	Paris	S 2	P 1	300
	P 3	Screw	Blue	17	Rome	S 2	P 2	400
	P 4	Screw	Red	14	London	S 3	P 2	200
	P 5	Cam	Blue	12	Paris	S 4	P 2	200
	P 6	Cog	Red	19	London	S 4	P 4	300
						S 4	P 5	400

The supplier-and-parts database

- who supply all part? List sname (hint, we need divide)
- because the constraint of unary and binary relation of dividend and divisor.

$$\pi_{SNAME}(\sigma_{P \#}(\pi_{SP} \div \pi_{P}))$$

- in any language, to be relational complete, we should be able to answer this.
- SQL?
- Who supply all parts that s2 supplies?

$$\pi_{SNAME}(\pi_{SP} \div (\pi_{P \#}(\sigma_{S \# = 'S2'}(\pi_{SP}))))$$

- SQL?

in order to do sql, we need relational calculus.

✓ Selection Operation (Ch 15.3)

- relational algebra select

Show some algorithm that use to implement relational algebra select



Only one operator, select

- we can have many algorithm to implement this
- select → pick row to satisfy the given condition
 1. (A1) Linear search (from first row, to last one)
 2. (A2) Clustering index, equality on key
 3. (A3) (clustering index, equality on non key)
 4. (A4)
 5. (A5)
 6. (A6)
 7.
- in the book, it look like we have 10 or more algorithm just for select operation.

(Chapter 12, 6th edition)

- if you have no index at all → the system may implement using linear search possible.
- if you have primary index,
 - but what is a primary index?



Selection Operation

- **File scan**
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
 - Cost estimate = b_r block transfers + 1 seek
 - ▶ b_r denotes number of blocks containing records from relation r
 - If selection is on a key attribute, can stop on finding record
 - ▶ cost = $(b_r/2)$ block transfers + 1 seek
 - Linear search can be applied regardless of
 - ▶ selection condition or
 - ▶ ordering of records in the file, or
 - ▶ availability of indices
- Note: binary search generally does not make sense since data is not stored consecutively
 - except when there is an index available,
 - and binary search requires more seeks than index search



Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (primary index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition
 - $Cost = (h_i + 1) * (t_r + t_s)$
- **A3 (primary index, equality on nonkey)** Retrieve multiple records.
 - Records will be on consecutive blocks
 - ▶ Let b = number of blocks containing matching records
 - $Cost = h_i * (t_r + t_s) + t_s + t_r * b$

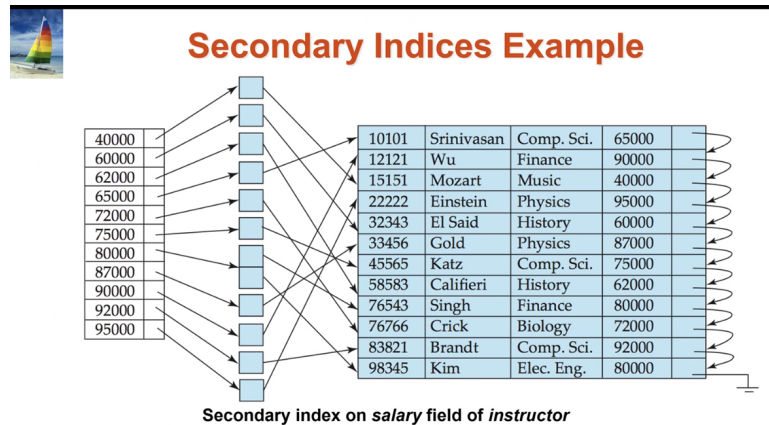
- key in database is based on the concept of identifier.
 - $S\#, P\#$ this are key.
 - use to identified particular row of the table.
 - key is logical
- index is access mechanism.
 - and are created outside of the table.
 - we use index to access table
 - index is physical.

What is primary index?

(Chapter 11, index and hashing)

index must be sorted, because if not you have to scan the whole thing to find

This is not primary index

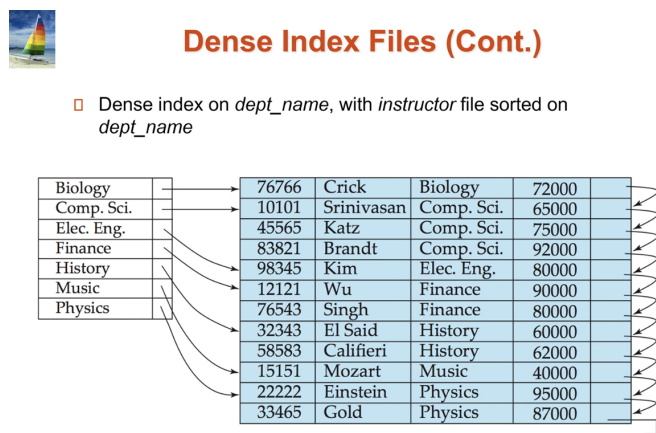


- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense

table sorted by id, not salary

- if the data is not sorted, it always secondary index.

The primary index



the index of this diagram is sorted by department, and the dataset is also sorted by department.

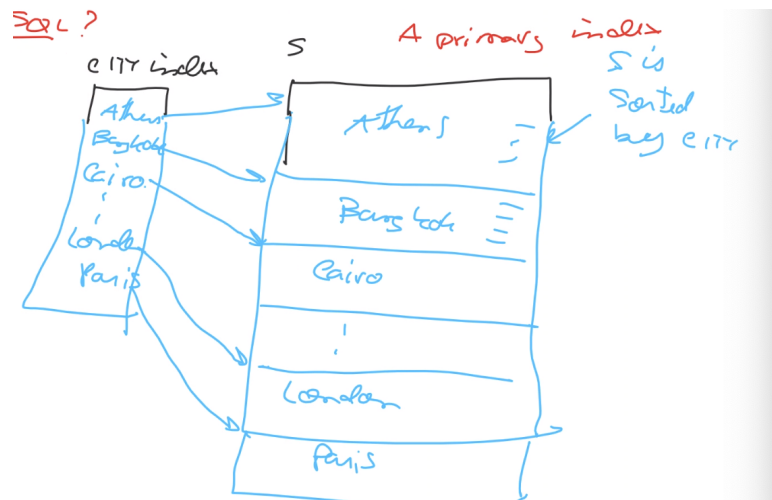
- so index and column is sorted by the same column.

Ordered indices. Primary Index

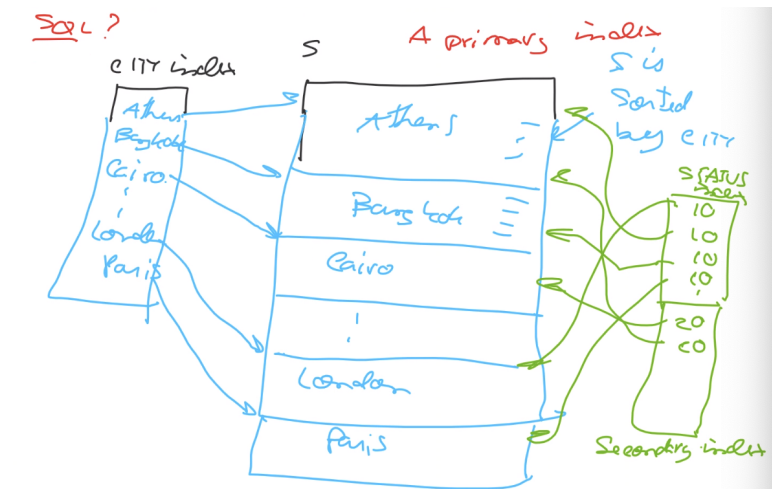


Ordered Indices

- In an **ordered index**, index entries are stored sorted on the search key value. E.g., author catalog in library.
- **Primary index**: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
 - Also called **clustering index**
 - The search key of a primary index is usually but not necessarily the primary key.



- city is primary index, where



- status is secondary index

index always sorted

- in relational db, logical level → row have no order, but in physical level → row have order.

S is physically sorted by city, we said that s is clustered by city.

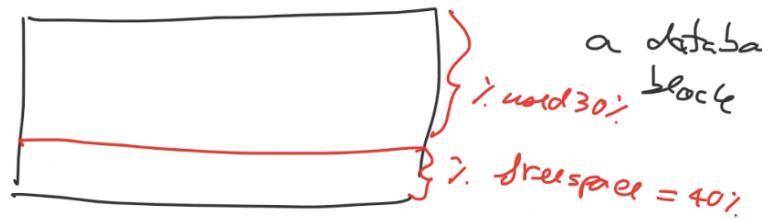
Select *
 From S
 where CITY = 'Paris'
 and STATUS = 30

- which is faster way?
- suppose city return 100 rows paris
- 30 return 20 rows.

from this, look like status is better, return less row.

- but look at other way, data is sorted by city, 100 paris is together. assume 1 db block contains 25 rows. this 100 rows require only 4 blocks.
- how many block it take for status 30 → but if have to estimate → use worst case scenatio → 1 row per block → it could be 20 blocks.
- therefore, using paris is better, even it return more rows.
 - so the fact that city is **primary index is an important factor**.

- when dbms collect db statistics → have to know block size, primary index,
- in practice, if you want best performance from the query that you use alot → just cluster based on some specific key.
 - but the price is we have maintain those sorted table physically.
 - export and import so that the row are together all the time with some space (it can overflow)



- when first initially load data to db space → leave 40% free space. (for insertion) (you can set)
 - after insert → the block can be full.
- if the row get delete under 30% → the row drop, get add to other block → so don't waste too much space.
 - in practice, we don't normally delete.